



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI

Dokumentacja projektu grupowego na przedmiot ZSD

Dokumentacja techniczna projektu

Wydział Elektroniki, Telekomunikacji i Informatyki

Politechnika Gdańska

Nazwa i akronim projektu: <i>CSAimBot, CSBot</i>	Zlecniodawca: dr hab. inż. Michał Czubenko	
Numer zlecenia: -	Kierownik projektu: Nazar Tomasz	Opiekun projektu: dr hab. inż. Michał Czubenko

Nazwa dokumentu/akronim: Dokumentacja techniczna projektu – DTP	Nr wersji: 1.0.1
Odpowiedzialny za dokument: Pietrzak Filip Nazar Tomasz Puczyński Karol	Data pierwszego sporządzenia: 7.04.2026
	Data ostatniej aktualizacji: 19.05.2026
	Studia I stopnia, inżynierskie
	<i>Semestr realizacji Projektu grupowego: 2</i>

Historia zmian

Wersja	Opis modyfikacji	Rozdział / strona	Autor modyfikacji	Data
1.00	Wstępna wersja	całość	Pietrzak Filip	7.04.2026
1.01	Wstępne opisanie aplikacji	2.3	Puczyński Karol	19.05.2026

Spis treści

1	Wprowadzenie - o dokumencie	2
1.1	Cel dokumentu	2
1.2	Zakres dokumentu	2
1.3	Odbiorcy	2
1.4	Terminologia	2
2	Dokumentacja techniczna projektu	2
2.1	YOLO oraz Dataset	2
2.1.1	Rozmiar datasetu	2
2.1.2	Struktura zdjęć	2
2.1.3	Opis zdjęć - YOLO	2
2.2	Robot – fizyczny i symulacja	2
2.2.1	Materiały i wykaz elementów POPRAWIC BO COS DOKUPOWALISMY2	
2.2.2	CoreXY – technologia	3
2.2.3	Symulacja - technologia	3
2.2.4	Screenshot z symulatora Gazebo	3
2.2.5	Modele 3D - technologia	4
2.3	Aplikacja i komunikacja z ESP32	4
2.3.1	Architektura Oprogramowania Aplikacji Głównej	4
2.3.2	Struktura klas i modułów	5
2.3.3	Architektura interfejsu graficznego oraz inicjalizacja systemu	6
3	Załączniki	7

1 Wprowadzenie - o dokumencie

1.1 Cel dokumentu

{Nie zmieniać}

Celem dokumentu jest udokumentowanie informacji dotyczących produktu, jego cech funkcjonalnych, parametrów technicznych, schematów blokowych, oprogramowania, wyników działania, zdjęć produktu, pomiarów, testów oraz innych elementów wymaganych przez opiekuna i klienta.

1.2 Zakres dokumentu

{Określenie, co wchodzi w zakres dokumentu, a co nie wchodzi, ew. wskazanie na dokumenty powiązane}

Dokument zawiera szczegóły techniczne projektu takie jak: rozmiary, technologie wykonania oraz szczegóły kodu. Dokument nie zawiera szczegółów na temat tego w jakiej kolejności powstały poszczególne części projektu oraz kto jest za nie odpowiedzialny – te opisy są raportowane na wykładach z zajęć z przedmiotu Zintegrowane systemy decyzyjne oraz na repozytorium na githubie

1.3 Odbiorcy

{Określenie adresatów dokumentu, może być to typ odbiorcy; tu: zleceniobiorca (Katedra), członkowie zespołu projektowego oraz wymienione z nazwiska osoby, do których dokument ma dotrzeć}

Odbiorcami projektu są dr hab. inż. Michał Czubenko oraz mgr inż. Jan Glinko.

1.4 Terminologia

{Wyjaśnienie używanych w dokumencie pojęć i skrótów, oznaczenia używane wewnątrz dokumentu np. oznaczenia wymagań}

Kierownik – kierownik zespołu - Tomasz Nazar

Zastępca kierownika – Filip Pietrzak

Dokument – Niniejszy dokument DTP

Zdjęcia z datasetu - screenshoty z gry Counter Strike: Global Offensive

CS:GO – oryginalna wersja gry Counter Strike: Global Offensive

eDPI - wartość uzyskana po przemnożeniu sensivity – ustawionej czułości w CS:GO – z DPI – ustawionej na komputerze czułości w myszce.

2 Dokumentacja techniczna projektu

{O zakresie dokumentacji decyduje opiekun, tu należy zacząć opis rozwiązania technicznego wg zaleceń opiekuna, w układzie redakcyjnym najlepiej oddającym charakter projektu – kolejne rozdziały, podrozdziały, punkty}

2.1 YOLO oraz Dataset

2.1.1 Rozmiar datasetu

Zdjęć do datasetu jest 10461.

2.1.2 Struktura zdjęć

Zdjęcia wykonywane są przy pomocy skryptu pythonowego w wielkości 1280x736 [px] dla Full HD oraz 1728x992 [px] dla 2K (wielokrotności 32 px) oraz skalowane do 1280x736px.

2.1.3 Opis zdjęć - YOLO

Opis zdjęć wykonywany jest w Roboflow, na dzień pierwszej wersji dokumentu (7.04.2026) zdjęć opisanych jest ponad 6 tysięcy.

2.2 Robot – fizyczny i symulacja

2.2.1 Materiały i wykaz elementów

1. Mechanika
 - Szyna liniowa MGN12 z wózkiem (40 cm) – 2 szt.
 - Szyna liniowa MGN12 z wózkiem (50 cm) – 1 szt.
 - Profil aluminiowy T-slot 2020 (40 cm) – 4 szt.
 - Narożniki aluminiowe (wersja 20) – 10 szt.
 - Pas zębaty GT2 (5 m, 10 mm szerokości) – 1 szt.
 - Rolki zębate GT2 – 6 szt.
 - Rolki gładkie GT2 – 2 szt.
 - Zębatki napędowe na wał silnika 8 mm – 2 szt.
 - Śruba trapezowa osi Z (skok 2 mm, 10 cm) – 1 szt.
 - Sprzęgło 8x5 mm – 1 szt.
 - Pręt stalowy 3 mm (długość 330 mm) – 2 szt.
 - Filament (TPU, 1,75mm, 1 kg) – 1 szt.
2. Silniki i sterowniki
 - Silnik krokowy Nema 23 (dwustronny wał 8 mm) – 2 szt.
 - Silnik krokowy Nema 17 "Pancake" – 1 szt.
 - Sterownik cyfrowy DM556 – 3 szt.
 - Enkoder obrotowy E6B2-CWZ6C (600 P/R) – 2 szt.
3. Chwytnik i klikanie
 - Serwo MG996R (180 stopni) – 1 szt.
 - Zębatka serwa 25T (Moduł 0.5) – 1 szt.
 - Listwa zębata (Moduł 0.5, 5x5x300mm) – 1 szt.
 - Solenoidy 12V – 2 szt.
4. Elektronika i sterowanie
 - Mikrokontroler ESP32 – 1 szt.
 - Konwerter poziomów logicznych TXS0108E (8-kanalowy) – 1 szt.
 - Przetwornica LM2596 ADJ (Regulowana) – 2 szt.
5. Zasilanie
 - Zasilacz (36V, 10A, 360W) – 1 szt.
6. Dodatkowe części drukowane
 - Chwytnik Fin Ray – 2 szt.
 - Podstawy do silnika – 2 szt.
 - Elementy wydrukowane (na Githubie)

2.2.2 CoreXY – technologia

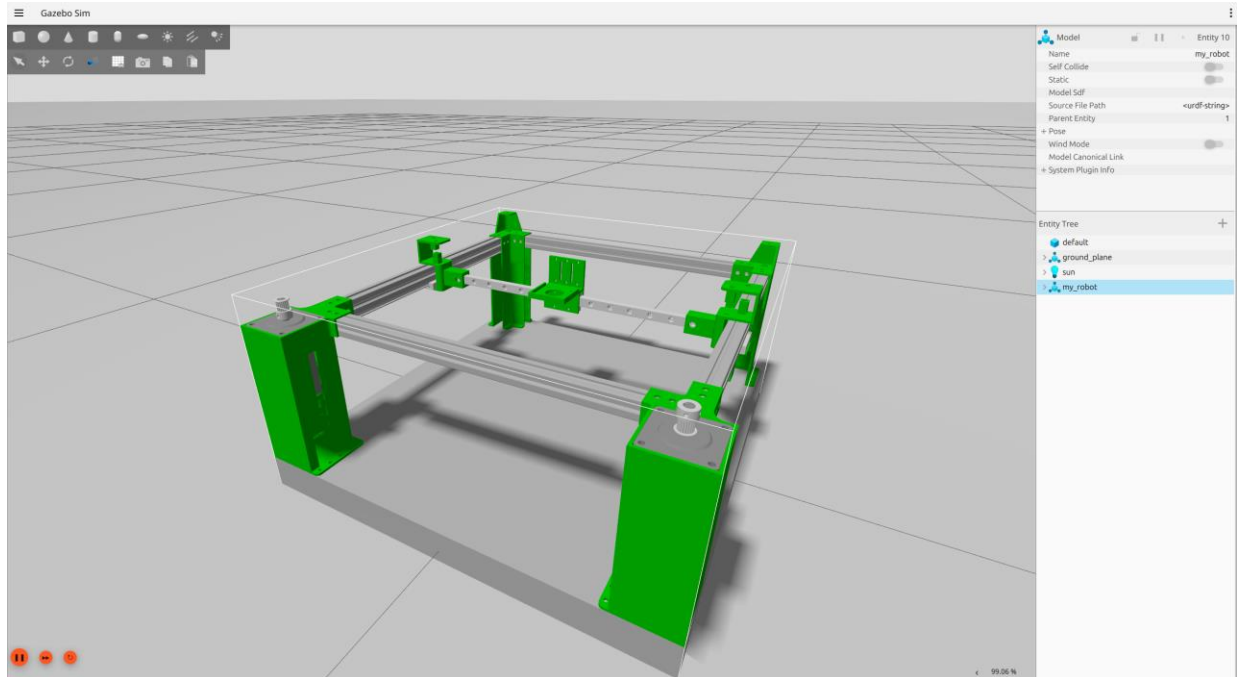
Wybór kinematyki CoreXY w projekcie CSAimBot jest podyktowany wymogiem uzyskania czasu reakcji poniżej 200 ms. Architektura ta opiera się na stacjonarnym montażu silników napędowych na

ramie głównej, podczas gdy ruch wózka realizowany jest za pośrednictwem układu pasów zębatych. Rozwiązanie to drastycznie zmniejsza masę ruchomą i inercję całego systemu.

2.2.3 Symulacja - technologia

Symulacja została zrealizowana przy użyciu środowiska ROS. Wirtualny model robota oraz fizyka otoczenia symulowane są w programie Gazebo, z kolei narzędzie RViz odpowiada za graficzną reprezentację danych z systemu sterowania.

2.2.4 Screenshot z symulatora Gazebo



2.2.5 Modele 3D - technologia

Modele 3D, projektowane w oprogramowaniu Autodesk Fusion, służą zarówno do stworzenia wirtualnej symulacji w środowisku Gazebo, jak i do fizycznej budowy robota kartezjańskiego z wykorzystaniem technologii druku 3D.

2.3 Aplikacja i komunikacja z ESP32

2.3.1 Architektura Oprogramowania Aplikacji Głównej

Aplikacja sterująca robotem CSAimBot została zaprojektowana w oparciu o architekturę wieloprotokółową. Rozdzielenie tych zadań na osobne procesy pozwala na ich równoległe wykonywanie i zapobiega blokowaniu interfejsu użytkownika podczas obciążających operacji wejścia/wyjścia lub obliczeń na GPU.

System składa się z jednego procesu głównego oraz dedykowanych procesów pomocniczych:

- **Proces Główny (*app.py*):** Punkt wejścia aplikacji. Inicjuje wszystkie procesy potomne, zarządza kanałami komunikacyjnymi. Proces ten centralizuje przepływ informacji – odbiera dane takie jak status z ESP32, czy potrzebny dystans do pokonania przez robota do trafienia w przeciwnika od procesów potomnych i dystrybuje do nich odpowiednie komendy (np. zmianę priorytetu celu).
- **Vision Worker (*aimbot.py*):** Niezależny proces odpowiedzialny za cały potok wizyjny. Realizuje asynchroniczne przechwytywanie klatek obrazu z ekranu, transfer danych bezpośrednio do pamięci VRAM oraz inferencję modelu do detekcji przeciwników. Wynikiem działania tego procesu jest ciągły strumień obliczonych przesunięć w osiach X i Y względem środka ekranu do wroga, przesyłany z określoną częstotliwością do procesu głównego.
- **Comms Worker (*comms.py*):** Proces odpowiadający za dwukierunkową komunikację sprzętową. Utrzymuje i monitoruje połączenie szeregowo z ESP32, asynchronicznie przysyłając ramki sterujące (zawierające m.in. wyliczone przesunięcia, stany przycisków myszy oraz prędkość) i odbierając statusy urządzenia.

- **Simulation Worker (*sim_worker.py*):** Proces odseparowujący logikę sterowania środowiska symulacyjnego od warstwy fizycznej robota. Zarządza komunikacją ze środowiskiem ROS2/Gazebo oraz symulowaną kinematyką układu.
- **Mouse Blocker Worker (*mouse_blocker_worker.py*):** Proces odpowiedzialny za programowe filtrowanie wejść z myszy fizycznej. Jego zadaniem jest zablokowanie dodatkowych przycisków i funkcji (takich jak przycisk środkowy, rolka), przepuszczając wyłącznie ruch oraz kliknięcia lewym i prawym przyciskiem myszy. Mechanizm ten uniemożliwia rejestrowanie przypadkowych akcji podczas fizycznego zaciskania chwytaka robota na obudowie myszy, co mogłoby zdestabilizować działanie programu. Proces ten aktywowany jest równolegle z procesem *Vision Worker*.

2.3.2 Struktura klas i modułów

W celu zapewnienia pełnej modułowości oprogramowania oraz separacji zadań, logika aplikacyjna podsystemu sterowania, komunikacji i detekcji została podzielona na niezależne moduły. Każdy z modułów odpowiada za realizację ściśle zdefiniowanego zadania i wykonuje się w odrębnym procesie. Poniżej przedstawiono szczegółowy opis struktury klas oraz metod wchodzących w skład struktury projektu.

Moduł wizyjny i inferencji (*aimbot.py*) Moduł ten stanowi rdzeń potoku przetwarzania obrazu. Odpowiada za realizację inferencji modelu YOLO w czasie rzeczywistym oraz wyznaczanie błędu między pozycją aktualną a zadaną.

- **Klasa AimBot:** Główna klasa zarządzająca potokiem wizyjnym. Odpowiada za konfigurację regionu przechwytywania, przygotowanie modelu YOLO oraz statyczną alokację zmiennych w pamięci GPU.
 - *prepare_camera()*: Definiuje statyczny obszar przechwytywania ekranu (FOV o rozdzielczości 1280x736 pikseli) dopasowany do środka rozdzielczości monitora głównego.
 - *prepare_model(model_path)*: Odpowiada za inicjalizację instancji sieci YOLO oraz wykonanie fazy rozgrzewkowej przy użyciu pustego tensora w pamięci CUDA w celu eliminacji opóźnień przy pierwszej klatce.
 - *allocate_variables()*: Dokonuje alokacji obiektów i tensorów w pamięci VRAM karty graficznej.
 - *capture_and_preprocess_frame()*: Pobiera surowy tensor z providera kamery, normalizuje wartości pikseli do zakresu [0.0, 1.0] i przepisuje dane bezpośrednio do tensora wejściowego modelu.
 - *calculate_best_target_position(boxes_data_tensor)*: Realizuje algorytm wyboru celu. Nadaje najwyższy priorytet klasom powiązanym z głową przeciwnika, a w przypadku ich braku przełącza się na klasy korpusu. W obrębie wybranej grupy priorytetowej selekcjonuje obiekt położony najbliżej środka ekranu i wylicza przesunięcie w pikselach.
 - *recoil_compensation(offset_x, offset_y)*: Wprowadza wirtualną poprawkę w osi pionowej symulującą przeciwdziałanie odrzutowi broni palnej, bazując na aktualnej broni.
 - *process_single_frame()*: Koordynuje pojedynczy cykl: wywołuje akwizycję obrazu, uruchamia inferencję sieci, aktualizuje stany wewnętrzne oraz opcjonalnie nakłada warstwę diagnostyczną OpenCV (okno debugowania).
- **Funkcja *vision_worker(pipe_conn, model_path, target_fps)*:** Funkcja uruchomieniowa procesu Vision Worker. Kontroluje taktowanie pętli przetwarzania, nasłuchuje komend zmiany parametrów z kanału komunikacyjnego (np. zmiana priorytetu celu na TT/CT/ALL) i asynchronicznie przesyła wyliczone przesunięcia do procesu głównego.

Moduł akwizycji obrazu (*camera.py*) Warstwa abstrakcji sprzętowej izolująca platformę systemową od mechanizmu pobierania klatek obrazu.

- **Klasa CameraProvider:** Zapewnia kompatybilność wieloplatformową potoku wizyjnego.
 - *__init__(region)*: Automatycznie wykrywa system operacyjny hosta. Na platformie Windows inicjalizuje bibliotekę *bettercam* wykorzystującą interfejs *NVIDIA Direct3D Desktop Duplication API*. Na platformach Linux uruchamia moduł *mss*.

- `grab_gpu_tensor()`: Pobiera aktualny zrzut zdefiniowanego regionu ekranu i natychmiast mapuje go do struktury tensora PyTorch bezpośrednio w pamięci karty graficznej, całkowicie omijając pamięć RAM i narzut procesora.
- `release()`: Zwalnia zasoby systemowe powiązane z przechwytywaniem obrazu.

Moduł komunikacji szeregowej i wejścia (*comms.py*) Odpowiada za warstwę transportową danych pomiędzy środowiskiem uruchomieniowym Python a mikrokontrolerem ESP32 oraz za przechwytywanie fizycznych naciśnięć klawiszy.

- **Klasa *SerialCommsModule***: Implementuje asynchroniczną komunikację UART poprzez port USB.
 - `connect()`: Otwiera port szeregowy z prędkością transmisji 115200 bodów i zarządza opóźnieniem wymaganym na restart układu po zestawieniu połączenia.
 - `check_connection(esp, port)`: Realizuje procedurę uwierzytelnienia. Wysyła zapytanie sekwencyjne "ESP32-CHECK" i oczekuje przez określony czas na flagę "ESP32-READY".
 - `send_command(command)`: Koduje tekstową ramkę sterującą do formatu UTF-8, dodaje znak powrotu karetki (`\r`) zgodny z protokołem i wymusza opróżnienie bufora wyjściowego.
 - `get_response()`: Odczytuje nadchodzące pakiety zwrotne z ESP32, dekoduje je i usuwa znaki niedrukowalne.
- **Klasa *KeyboardInputModule***: Niskopoziomowy wątek nasłuchujący zdarzeń klawiatury systemowej za pomocą biblioteki `pynput`.
 - `on_press(key) / on_release(key)`: Dodaje lub usuwa z wewnętrznej struktury zbioru zdefiniowane klawisze sterujące, eliminując problem powtórzeń systemowych.
 - `get_key()`: Przetwarza stan wciśniętych klawiszy. W pierwszej kolejności weryfikuje wystąpienie klawisza wyłączenia awaryjnego, a w przypadku jego braku generuje ciąg znaków reprezentujący aktualny wektor manualnego sterowania osiami.
- **Funkcja *comms_worker(conn)***: Funkcja zarządzająca procesem *Comms Worker*. Agreguje dane z potoku IPC interfejsu graficznego, wysyła sformatowane ramki sterujące (zawierające offsety, prędkość, stany przycisków myszy) z zachowaniem bezpiecznego interwału czasowego i przekazuje zwrotne logi diagnostyczne z ESP32 do interfejsu GUI.

Moduł filtrowania przycisków myszy (*mouse_blocker_worker.py*) Realizuje zadania związane z izolacją fizycznych peryferiów od działań systemu automatycznego sterowania.

- **Funkcja *mouse_blocker_worker(pipe_conn)***: W zależności od wykrytej platformy operacyjnej wywołuje dedykowane funkcje blokujące:
 - `_run_windows_blocker()`: Wstrzykuje filtr zdarzeń biblioteki `pynput` do kolejki komunikatów systemu Windows. Przechwytuje i blokuje kody komunikatów powiązane ze scrollmem, przyciskiem środkowym oraz przyciskami bocznymi, zapobiegając ich rejestracji przez system operacyjny.
 - `_run_linux_blocker()`: Przejmuje bezpośrednią kontrolę nad deskryptorem pliku fizycznej myszy w podsystemie `evdev` za pomocą metody `grab()`. Tworzy wirtualny interfejs wejściowy *UInput*, na który przekazuje wyłącznie surowe zdarzenia relatywne ruchu oraz podstawowe stany kliknięć (`BTN_LEFT`, `BTN_RIGHT`), całkowicie odcinając pozostałe przerwania sprzętowe.

2.3.3 Architektura interfejsu graficznego oraz inicjalizacja systemu

Zwieńczeniem architektury oprogramowania jest proces główny, który integruje wszystkie procesy pomocnicze i zapewnia interfejs graficzny. Został on zrealizowany przy użyciu biblioteki *Dear PyGui*.

Punkt wejścia i inicjalizacja (*app.py*) Skrypt startowy aplikacji, odpowiedzialny za bezpieczne powołanie do życia środowiska wieloprotocowego.

- Wykorzystuje bibliotekę *multiprocessing* do utworzenia niezależnych kanałów łączności dla każdego z workerów (wizyjnego, komunikacyjnego, symulacyjnego oraz blokady myszy).
- Odpowiada za zarządzanie cyklem życia procesów potomnych – ich poprawne uruchomienie, a w przypadku zamknięcia aplikacji (lub wystąpienia błędu krytycznego) – przesłanie sygnałów terminacji oraz zwolnienie zasobów za pomocą metody `join()`.

- Obsługuje dodatkowe flagi uruchomieniowe (np. --export-only), które pozwalają na wywołanie skryptu w trybie kompilacji przestrzennej modelu YOLO do zoptymalizowanego formatu TensorRT (silnik .engine) przed właściwym startem aplikacji.

Klasa interfejsu użytkownika (gui.py) Zarządza warstwą graficzną oraz dwukierunkową wymianą danych za pośrednictwem dedykowanych potoków komunikacyjnych.

- **Zarządzanie stanem i nawigacja:** Interfejs opiera się na wielozakładowym układzie z rozwijanym panelem bocznym. Przełączanie poszczególnych widoków (Strona Główna, Panel Sterowania, Symulacja, Statystyki, Ustawienia) realizowane jest poprzez manipulację właściwością widoczności odpowiednich grup widżetów.
- **Responsywność i skalowanie:** Aplikacja natywnie obsługuje rozdzielczości 720p oraz 1080p.
- **Wielojęzyczność:** Interfejs posiada wbudowany słownik, który mapuje etykiety tekstowe i opisy na wybrane warianty językowe (polski/angielski).
- **Wizualizacja telemetrii:** Aplikacja wykorzystuje wbudowane silniki renderowania wykresów do prezentacji danych w czasie rzeczywistym. Generowane są wykresy historii pozycji robota w osiach X i Y oraz dystansu do wykrytego celu.
- **Asynchroniczna pętla zdarzeń:** Aby uchronić główną pętlę renderującą Dear PyGui przed zablokowaniem, odbiór komunikatów od procesów pomocniczych został oddelegowany do osobnego wątku. Wątek ten w nieprzerwanej pętli odpytuje potoki za pomocą metody i w przypadku wykrycia nowych pakietów danych (np. statusu połączenia, logów UART z ESP32) aktualizuje odpowiednie zmienne stanu aplikacji oraz powiązane widżety tekstowe.

3 Załączniki

{Wszelkie dokumenty nie dające się wkomponować w prosty sposób w tekst, należy dołączyć w osobnych plikach, a ich spis przedstawić w formie tabeli, przykładowo:}

Tabela 3.1. Lista załączników

L.p.	Nazwa dokumentu	Nazwa pliku
1.	Schemat funkcjonalny	
2.	Schemat blokowy	
3.	Schemat ideowy	
4.	Wykaz elementów	
5.	Karta katalogowa, itp.	
6.	Dokumentacja tworzona za pomocą innego narzędzia	